LSM Trees — From First Principles

Published: 2026-10-14

Database (/tag/database?source=post_page---header_tags--3dcd38d6ff76----------)

Lsm Tree (/tag/lsm-tree?source=post_page---header_tags--3dcd38d6ff76----------)

Storage Engine (/tag/storage-engine?source=post_page---header_tags--3dcd38d6ff76----------)

Why would anyone even need to study a database storage engine or an LSM Tree, was the first question I stumbled upon when I saw someone make their own storage engine on X or Twitter.

Back then, I did not have the answer for it. Now I do.

Because we want our applications or projects to work beautifully and serve their needs using the tools we currently possess. Now, no tool is wrong to use, but each has its use case and to understand which tool works where **IS** what we need so we don't end up like *Patrick* did:



Here we'll be looking at a *log-structured* storage engine called **LSM**(Log-Structured Merge Trees) which uses an immutable file-storage system.

Before understanding LSM we shall do a mental exercise of creating our own database. The simplest database that can mimic LSM's property of being *immutable*.

```
get(...args) {

}

set(...args) {

}
```

The simplest database will have two APIs or functions: *get()* to get the data and *set()* to set the data.

To save the data we'll store the key-value pair in a text-file separated by a comma(,) and each call to *set()* will append the data to the end of the file.

Before moving ahead can you guess the problem with these functions?

Nothing to sweat about if you cannot.

The time complexity for *set()* is O(1) because its simply appending the key-value pair to the end of the file.

The time complexity for *get()* is O(n) because its searching the whole file for the existence of a single key. When, starting out with a small number of entries, it won't matter but as *set()* appends more and more keys to the file, querying for a single key will become expensive.

Can you suggest something that could speed up reads?

If you said, *indexes* you hit the jackpot.

Indexes are another form of data structure used to optimize reads. Its a look-up table updated automatically. Many databases allow to add or remove indexes as per our need.

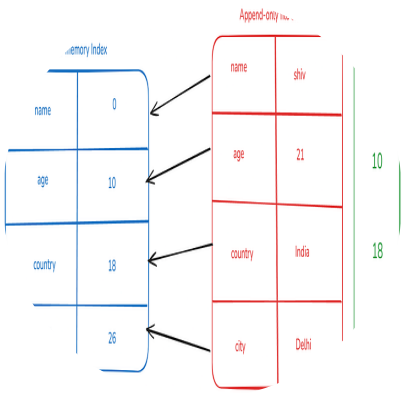
The downside of using indexes is that write becomes heavy, because now we've to update two places instead of one. Hence not every part of the database is supposed to be indexed.

Whenever we append to a file the OS knows the current file position. For example if the current position is at 37 bytes, then appending

age = 23

will happen exactly at byte 37.

Using the above fact and some DSA fundamentals we can arrive at an approach where we use a *hashmap* to store keys with their respective byte offsets. A hashmap has an O(1) time complexity for reads and when looking for a key we can use the hashmap to jump directly to the value.



One could argue, that we can instead store the key value pair directly inside the hashmap instead of byte offsets. You're right, until you've to store large blobs of data which can lead to RAM exhaustion.

Even with using indexes there are a lot of things to deal with.

1. We keep appending to a file and not clearing outdated entries. We might run out of disk space.
2. The index hashmap exists only in RAM. Its destroyed and built from scratch on whenever we restart our database.
3. To build the hashmap from scratch we need to scan the whole file and find the byte offset for each key. This can become a hectic process if we've too many entries.
4. To solve the above problem we can use an on disk hashmap. But, an in-memory hashmap is difficult to maintain. There's a lot of random access I/O and difficult to grow them when they become full. Also hash collisions are something to be taken care of.
5. Currently we cannot support range queries.

In practice, actual databases don't use hashmaps but sorted keys (https://www.evanjones.ca/ordered-vs-unordered-indexes.html).

So, much for building our own database.

SSTable and Memtable

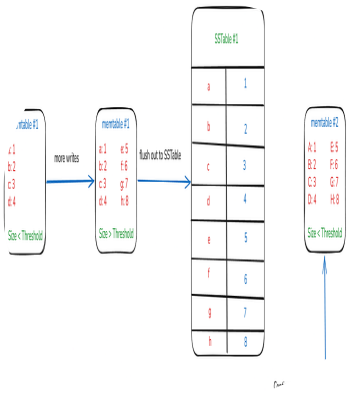
To reach a common ground between our approach and the one being used in enterprise, production-grade databases we use a hybrid approach of appending in a sorted manner.

Whenever a write comes in, we'll append directly to an in-memory data structure called *memtable*, which will either be a trie, skip list or a red-black tree. These data structures allow efficient and sorted reads and data can be inserted into them in any order.

When the size of memtable will grow above a certain threshold, we'll flush out the contents of the memtable onto an immutable file on the disk called *SSTable*. This SSTable or Sorted Strings Table file will contain the most recent segment of the database and will be placed along side many other such SSTables which will be storing older segments of the database. Each table will have its own index. While the new segment is being written out the database can carry on appending to a new memtable instance and the old memtable's memory will be freed when writing of the SSTable has been completed.

When trying to read the value of a key, we first try to find the key in the memtable and the most recent on-disk segment. If it's not there, we keep looking in the next-older segment until we either find the key or reach the oldest segment. If the key does not appear in any of the segments, it does not exist in the database.

To ensure that the data in the memtable is not lost if the database crashes, the storage engine keeps a separate log on disk to which every write is immediately appended. This log is not sorted by key, because its only purpose is to restore the memtable after a crash. Every time the memtable gets written out to an SSTable, the corresponding part of the log can be discarded.

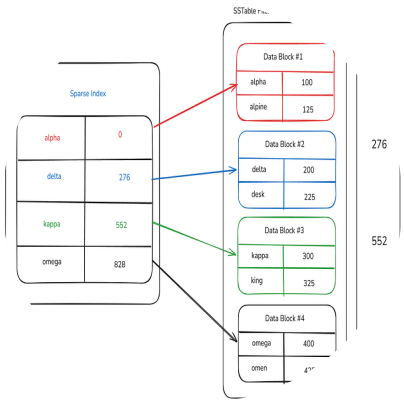


An SSTable stores key-value pairs in a sorted order in blocks of a few kilobytes and then stores the first key of each block in an index. This type of index is called a *sparse* index, since it only stores a small number of keys. This sparse index is made up of immutable B-Trees or tries and lives in a separate part of the table.

Reason why we make an SSTable immutable is because:

1. **Random writes are slow.** Disks or even SSDs are faster when you write things sequentially instead of jumping around and poking at random spots. Editing in-place means jumping to different places and tweaking a few bytes. That's like asking someone to edit random words scattered across a 500-page book instead of letting them write a fresh page at the end.
2. **What if someone's reading the file while you're editing it?** What do they see? Half-old, half-new data? Garbage? To prevent that we need locks — "nobody read this file I'm still writing to it". Locks slow everything down and make things more complicated more easily.

Having immutable segment files also simplifies crash recovery. If a crash happens while writing out the memtable or while merging segments, the database can just delete the unfinished SSTable and start afresh.



Finding a key in a SSTable

For instance, let's say you're looking for **king** inside the table, which doesn't appear in the sparse index. Because of sorting, you know **king** must appear between the keys **kappa** and **omega**. You can seek the offset to **kappa** and look for **king**, if present. Scanning blocks of few kilobytes can be done quickly.

We've been pushing a lot on this "sorted" approach. If sorting is so good why not we always keep one long sorted file. But we cannot because SSTables are optimized for faster reads, since writes are difficult. We simply cannot append to the end of the file which would violate the whole approach of "sorted strings" or "sorted keys". And, reconstructing the

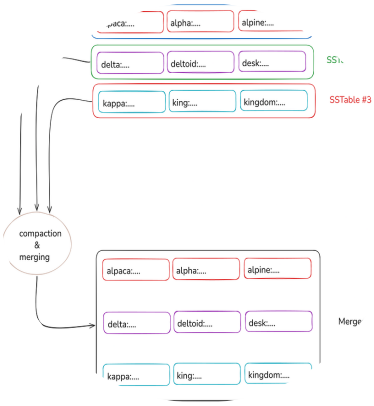
whole table from scratch will be more expensive.

By continuing the above process we'll run into multiple SSTables, some having live versions of the database, some with older and outdated versions. To solve this problem we do *merging and compaction.*

Merging and Compaction

From time to time, run a merging and compaction process in the background to combine segment files and to discard overwritten or deleted values.

Merging segments works similarly to the *mergesort* algorithm. We start by reading the input files side by side, looking at the first key in each file, copy the lowest key (according to the sort order) to the output file, and repeat. If the same key appears in more than one input file, we keep only the more recent value. This produces a new merged segment file, also sorted by key, with one value per key, and it uses minimal memory because we can iterate over the SSTables one key at a time.



An important detail about LSMs is how compaction is done. Many LSM storages allow us to define our compaction strategies. Common choices are:

Size-tiered compaction

Newer and smaller SSTables are merged into older and larger SSTables. The SSTables containing older data can get very large and merging them requires a lot of temporary disk space. However, this kind of compaction is built to handle heavy writes since merge is done occasionally.

Imagine keeping a big stack of papers and stapling them together when enough papers of similar size pile up. This can lead to old papers just sitting there waiting for their turn. Exactly the same happens with the old segments, which can lead to key overlaps — file A might have keys from 1–100 while file B might also have 50–150. No rule preventing overlap. Hence, this kind of compaction is *write-friendly* since writing is simple: memtable fills up -> flush into new SSTable. Done. Heavy weightlifting for merging is done occasionally in big batches.

Since keys overlap, frequently and stale duplicates may survive a long time up until a merge happens, a single read might have to check multiple SSTables before confirming if the key exists or not.

In size-tiered, compaction only happens when a whole tier fills up (e.g., 4 files of the same size). Until that trigger fires, all those stale, duplicate, dead versions of keys just accumulate on disk, doing nothing but wasting space. And on top of that — during the merge itself, you need room for *both* the old files and the new merged file simultaneously (you can't delete

the old ones until the new merged one is safely written). So briefly, you need up to ~2x the space of what's being merged.

Level-tiered compaction

Instead of rewriting large SSTables, we can have different levels of storage for SSTables where L0 has the most recent segment. Each SSTable has a fixed size and is grouped into different levels. All levels beyond L0 have key range partitioned tables. Let's say L1 can have two tables: one ranging from **a-m** and another **n-z.** Each level has its fixed size and each level has a size greater than the one before it i.e. L2 will have a bigger size than L1.

Here, compaction happens incrementally and uses less disk space as compared to size-tiered compaction. It is more read-optimized since there are only a few tables to read from.

Since ranges don't overlap within a level, there's at most *one* file per level that could possibly contain my key. With a handful of levels (maybe 5–7 even for huge datasets), that's a small, bounded number of files to check — much better than size-tiered's "could be scattered anywhere."

That's why leveled is *read-optimized*.

To *keep* those non-overlapping ranges as new data keeps flowing in, you can't just dump new files wherever — you have to constantly push data down: when a level gets too big, take a file, merge it with whatever overlaps in the next level down, and rewrite that chunk. This happens continuously, in small pieces, on basically every flush cycle. That's a lot more total rewriting of the same data over its lifetime — more write amplification — which is why it's less write-friendly than size-tiered's "just append and batch-merge occasionally" approach.

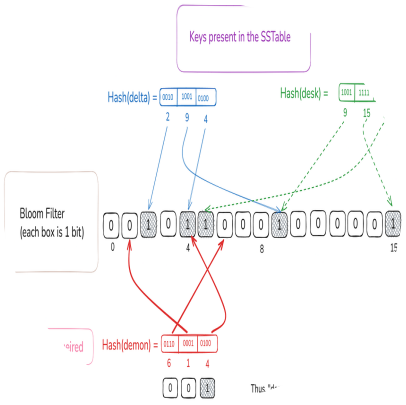
Since stale data gets cleaned up continuously as part of these small ongoing merges (rather than waiting for a big pile-up), leveled keeps much less garbage lying around — so its space amplification stays low and predictable.

So the whole tradeoff in one line:
Size-tiered delays and batches its cleanup work → **cheap writes**, but messy overlapping files → **expensive reads** and **more wasted space**. **Leveled** does cleanup constantly in small pieces → keeps things tidy for **fast, bounded reads** and **low space usage**, but pays for that tidiness with more total rewriting on the write path.

Bloom Filters

Reading a key in an LSM storage is slow since the key needs to checked in multiple segment files to be certain that the key exists or it does not. To solve this problem we use **Bloom Filters.**

We keep a bloom filter for each SSTable instead of one large bloom filter. The reason is that one large bloom filter can only tell us that a particular key might exist in the system — which is completely useless since we still don't know which SSTable file to open.



For every key in the SSTable we calculate a hash function which produces some numbers, which are then used as indexes for the array of bits. Let's take an example of a Bloom filter containing two keys and 16 bits (in reality, it would contain more keys and more bits). The key *delta* hashes to the numbers (2, 9, 4), so we set the second, ninth, and fourth bits to 1. The bitmap is then stored as part of the SSTable, along with the sparse index of keys. This takes a bit of extra space, but the Bloom filter is generally small compared to the rest of the SSTable.

When we want to look for another key say, *demon* the key hashses into the numbers (6, 1, 4). One of these bits is 1 (bit number 4), while the other two are 0. Checks can be made extremely quick using bitwise operations that all CPUs support.

If even one of the hash numbers is 0 we know that the key does not exist but if all numbers are 1, there's a possibility that the key exists or coincidentally all those bits were set to 1 by other keys. This leads to a case of where a key might not be present in the SSTable but it looks like it does is called *false positive.*

The probability of a false positive depends on the **number of keys**, the **number of bits set by the keys**, and the **number of bits in the bloom filter.** We need to allocate 10 bits of bloom filter space for every key in the SSTable to have a probability of 1% false positive cases, and the probability reduces by ten-fold for every 5 additional bits we allocate per key.

If Bloom Filter says the key is not present we can safely skip that SSTable, since we're sure the key does not exist.

If Bloom Filter says the key exists, we need to consult the sparse index to find the correct data block and check whether the key exists in that data block or not. If it was a false positive case, we've done something unnecessary work but otherwise no harm done, and we continue checking in the older segment.

Deleting a key

If you want to delete a key and its associated value, you have to append a special deletion record called a *tombstone* to the data file. When log segments are merged, the tombstone tells the merging process to discard any previous values for the deleted key. Once the tombstone is merged into the oldest segment, it can be dropped.

Every design decision we've made so far — **memtable, SSTable, sparse index, tombstones, background merging** exists for one reason — **to make writes as cheap as possible by deferring the expensive, disk-heavy work (sorting, deduplicating, reclaiming space) to the background.** That's the whole bet an LSM tree makes. The cost of that bet is paid on the read side: a lookup might have to check several places before it finds — or fails to find — a key. That's why LSM trees are the storage engine of choice for write-heavy systems (Cassandra, RocksDB, HBase).

References: DDIA by Martin Klepmann.